

Kyndryl Fabric [K360] Enterprise Transformation Platform

Architecture & Technical Design Document

Version: 3.0

Date: March 2026

Audience: CIO, CTO, Enterprise Architecture

Classification: Confidential — Technical Reference

1. Executive Summary

The Kyndryl Fabric [K360] Enterprise Transformation Platform is an autonomous, AI-native governance system designed to close the gap between strategic intent and operational execution across large-scale transformation portfolios.

Traditional enterprise dashboards aggregate data. Kyndryl Fabric [K360] reasons about it.

The platform deploys nine autonomous AI agents that continuously analyze portfolio data across financial, operational, compliance, and change management domains. These agents operate on their own cadence — detecting anomalies, generating executive insights, collaborating with peer agents through a structured Agent-to-Agent (A2A) protocol, and recommending interventions — without waiting for human initiation.

What Makes This Architecture Distinct

1. Autonomous Multi-Agent Intelligence — Nine domain-specialist agents (Value Realization, Strategic PMO, Transformation, FinOps, OKR, Governance, Planning, OCM, Notification) run continuously on independent analysis cycles. They don't just respond to queries — they proactively identify risks, surface opportunities, and trigger cross-agent collaboration when issues span multiple domains.

2. Continuous Autonomous Learning (Mem0 + Letta) — Agents maintain dual-layer cognitive memory — short-term episodic recall (Mem0) and long-term institutional knowledge (Letta) — creating a closed-loop learning system where each analysis cycle builds on every previous one. Agents develop domain expertise over time, recognize recurring patterns across hundreds of cycles, and learn from human feedback on interventions.

3. Policy-as-Code Governance with Vision AI — Enterprise SOPs, regulatory filings, and compliance frameworks are ingested using Qwen/Qwen3-VL-32B, a vision-language model that processes documents as images — preserving table structures, flowchart logic, approval matrices, and scanned signatures. Extracted rules are transformed into machine-executable YAML and enforced through GoRules decision tables and Rulebrick tool execution rules.

4. Agent-to-Agent (A2A) Protocol — A structured communication protocol enables agents to delegate tasks, share findings, and conduct multi-agent investigations. When the FinOps agent detects a budget anomaly with schedule implications, it initiates a formal A2A request to the Strategic PMO agent, receives a correlated analysis, and presents a unified finding to leadership.

5. Deterministic + Probabilistic Governance — A GoRules decision engine with 42 configurable threshold rules provides fast, deterministic monitoring (millisecond response, zero API cost). The AI agents provide qualitative analysis and reasoning. Together, they form a governance system that is both reliable and intelligent.

6. Bidirectional Integration Fabric — A Model Context Protocol (MCP) server synchronizes data with 12 external PPM tools (Jira, OpenProject, Confluence, ServiceNow, Azure DevOps, and others). The system doesn't replace existing tools — it governs across them. Changes flow in both directions through a structured write-back API.

7. What-If Simulation with Agent Re-Run — A simulation sandbox lets leadership modify portfolio variables (budgets, timelines, resources) and re-run the AI agents against the modified data to preview how changes propagate across the portfolio before committing them to production.

8. Agentic Backbone via ACP + A2A — Kyndryl Fabric [K360] operates as an enterprise Agentic Backbone using Google's A2A and IBM/BeeAI's ACP protocols, allowing external agents (from ServiceNow, SAP, Salesforce, or custom platforms) to discover K360 agents, delegate tasks, and participate in multi-platform workflows.

The platform is built as a full-stack TypeScript monorepo: React 18 frontend, Express.js backend, PostgreSQL database (105 tables, Drizzle ORM), with all AI capabilities powered by Anthropic Claude through the Mastra agent framework.

2. Architecture Overview

2.1 Layered Architecture

The system follows a seven-layer architecture with clear separation of concerns. Data flows upward from external systems through the integration and storage layers, is normalized by the Ontology Layer (SAFe 6.0 canonical model), enriched by the Semantic Layer (Neo4j Temporal Knowledge Graph), analyzed by the intelligence layer, and presented through the dashboard. Agents communicate via two complementary paths: the A2A protocol for direct peer-to-peer task delegation and messaging, and the Temporal Knowledge Graph for shared context, causal reasoning, and cross-agent knowledge discovery.

Kyndryl Fabric [K360] — Seven-Layer Architecture

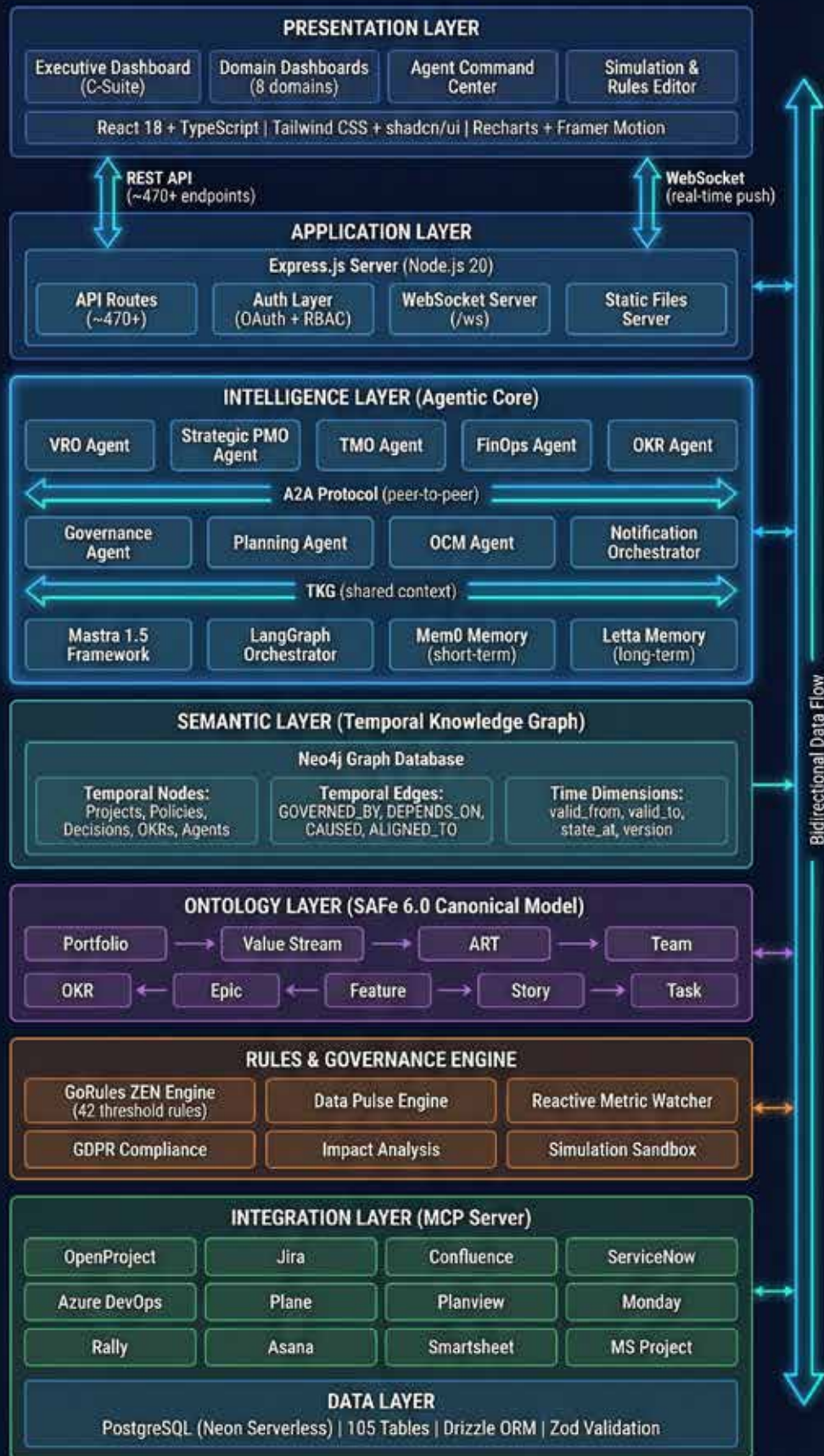


Figure 1: Seven-Layer Architecture of Kyndryl Fabric [K360]

2.2 Data Flow Architecture

Data flows bidirectionally through the platform. External PPM systems feed into the MCP Integration Layer, which normalizes all data to the SAFe 6.0 ontology before persisting it in PostgreSQL. The Temporal Knowledge Graph enriches this data with time-versioned relationships. The dual governance engines — deterministic rules and probabilistic AI agents — analyze the enriched data and push results to the React 18 dashboard via WebSocket.

Kyndryl Fabric [K360] — Data Flow Architecture

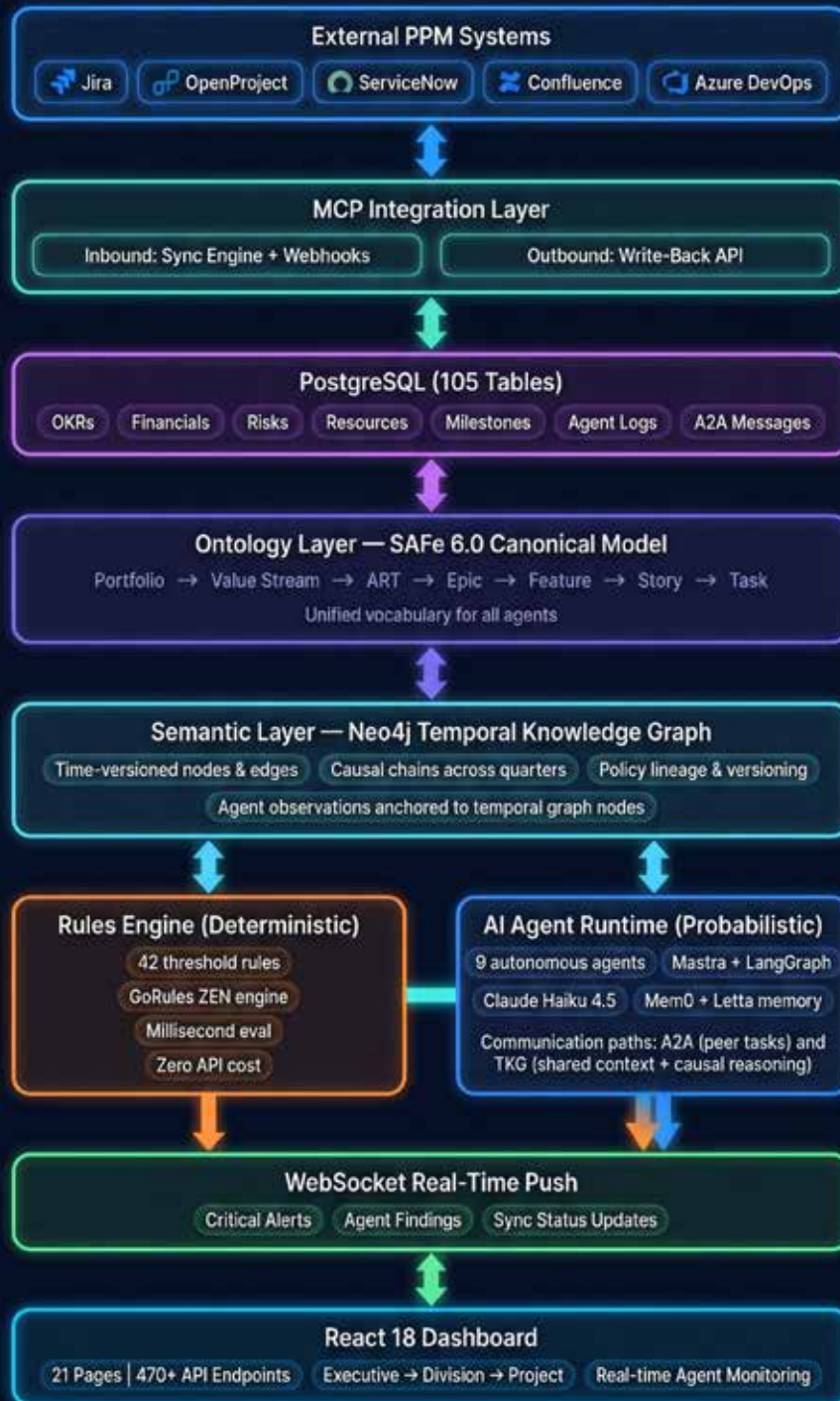


Figure 2: Bidirectional Data Flow Architecture

3. The Agentic Core — What Makes This Architecture Unique

3.1 Why Multi-Agent Architecture

Most enterprise dashboards present static or periodically refreshed data. Analysts manually review metrics, identify anomalies, investigate root causes, and recommend actions. This process is slow, inconsistent, and doesn't scale across a portfolio of 20+ programs spanning multiple business units.

Kyndryl Fabric [K360] replaces this manual analysis loop with nine autonomous agents that operate continuously — each specializing in a domain that would traditionally require a dedicated analyst or governance function. The agents don't wait for someone to ask "what's wrong?" — they proactively surface findings, correlate signals across domains, and recommend specific actions with supporting evidence.

The decision to use multiple specialized agents rather than a single monolithic AI was deliberate. A single agent analyzing everything would require massive context windows, produce unfocused findings, and lack the domain-specific expertise needed for nuanced portfolio governance. Specialized agents with defined scopes produce more actionable, precise findings — and can collaborate with peer agents when issues span domains.

3.2 The Nine Autonomous Agents



Figure 3: The Nine Autonomous Agents and Their Domains

Agent	Domain	Autonomous Capabilities
-------	--------	-------------------------

Value Realization (VRO)	Value & ROI	Continuously tracks ROI and NPV across investment portfolios. Monitors benefit realization timelines against targets. When ROI drops below threshold, initiates A2A collaboration with FinOps for budget correlation and PMO for schedule impact assessment.
Strategic PMO	Portfolio Delivery	Replaces traditional Gantt-chart monitoring with Lean/Agile flow metrics (cycle time, throughput, WIP). Detects schedule slips by analyzing velocity trends rather than date comparisons — identifying problems weeks before they surface in milestone reports.
TMO (Transformation)	Change Programs	Manages transformation program oversight. Tracks adoption curves against targets, correlates initiative progress with business outcome metrics, and flags transformation fatigue when adoption velocity declines across business units.
FinOps	Financial Operations	Monitors budget vs. actual spend across the portfolio. Detects cost anomalies, generates forecasts based on burn rate trends, and recommends cost optimization opportunities. Collaborates with VRO to validate that spending aligns with value delivery.
OKR Alignment	Strategy Execution	Ensures strategic alignment from company OKRs through division objectives to project-level key results. Calculates alignment scores, detects orphaned projects (work not linked to any strategic objective), and monitors key result progress against quarterly targets.
Governance Guardian	Compliance & Risk	The compliance watchdog. Monitors regulatory compliance checkpoints, policy adherence, and governance framework conformance. Generates audit trail entries for all significant decisions. Flags governance gaps before they become audit findings.
Strategic Planning	Capacity & Roadmap	Coordinates capacity forecasting, resource optimization, and dependency analysis across program increments. Runs scenario models to evaluate the impact of prioritization decisions on the overall portfolio timeline.
OCM Readiness	Change Management	Manages the people side of transformation. Tracks stakeholder readiness assessments, training completion rates, communication effectiveness, and adoption barriers. When readiness scores drop, initiates A2A collaboration with TMO to adjust change strategies.
Notification Orchestrator	Communication	Central hub for all agent-generated signals. Receives alerts from every other agent, deduplicates overlapping findings, prioritizes based on severity and organizational impact, and routes notifications to appropriate stakeholders.

3.3 Autonomous Runtime — How Agents Think and Act

The Agent Runtime is the heartbeat of the platform. It runs as a background process that manages agent execution cycles, memory management, and inter-agent communication. Every three minutes, the runtime selects agents to execute, assembles their context from live database data and recalled memories, invokes Claude for analysis, and processes the structured JSON results.



Figure 4: The Four-Stage Agent Runtime Cycle (Every 3 Minutes)

Intelligent Resource Management

Inactivity Pause — If no users are connected via WebSocket for 10+ minutes, all agent cycles pause automatically. Agents resume instantly when a user reconnects. This prevents unnecessary API spend during off-hours.

Data-Change Gating — If no data has changed since the last cycle (no data pulse updates, no user edits, no sync events), the runtime skips the AI analysis and only performs deterministic metric calculations. This can reduce API calls by 40–60% during stable periods.

Round-Robin Selection — Only 3 agents run per cycle (VRO + 2 rotating). This distributes API load evenly and keeps per-cycle costs predictable while ensuring all agents complete a full analysis rotation within minutes.

3.4 Agent Orchestration — Three Coordination Patterns

The platform uses three distinct orchestration patterns, each optimized for different coordination scenarios:

Pattern 1: Centralized Task Queue (Orchestrator): The Agent Orchestrator maintains a priority-sorted task queue. When an event requires agent action (a threshold breach, a user request, or an external sync), the orchestrator evaluates which agent is best suited based on a capability registry and current workload. Conflict resolution follows: Priority > Agent Rank > Creation Time.

Pattern 2: Reactive Cascade (Event-Driven): Pre-defined cascade scenarios trigger automatic multi-agent workflows when specific actions occur. This is the "nervous system" — fast, predictable, and hardcoded for known scenarios. For example, a Budget Reduction Cascade automatically sequences through FinOps → VRO → TMO → Planning → Governance agents via a publish/subscribe mechanism.

Pattern 3: Autonomous A2A Collaboration (Peer-to-Peer): When an agent's LLM analysis identifies a cross-domain implication, it initiates a formal A2A request to the relevant peer agent. This is emergent collaboration — not pre-programmed, but determined by the agent's reasoning. For example, the VRO agent detecting ROI decline correlated with schedule delays autonomously requests a TMO impact assessment.

3.5 Agent-to-Agent (A2A) Communication Protocol

The A2A protocol operates on two layers:

Layer 1: Task-Based Protocol (Synchronous)

Used by the Agent Runtime for skill-based delegation. Each agent registers an Agent Card describing its identity and available skills:

Agent	Registered Skills
VRO	investment_recommendations, budget_alerts, scenario_modeling
Strategic PMO	flow_metrics, milestone_alerts, resource_allocation
FinOps	cost_optimization, budget_forecasting, spend_analytics
Governance	compliance_check, policy_validation, audit_trail
Synthesizer	cross_domain_analysis (default router for complex queries)

Layer 2: Persistent Messaging Protocol (Asynchronous)

For ongoing multi-agent conversations, the platform maintains a persistent messaging system backed by PostgreSQL:

Message Type	Purpose	Example
query	Request data or analysis	"What is the current CPI for Division Y?"
response	Reply to a query	"CPI is 0.87, trending down from 0.92 last quarter"
insight	Proactive knowledge sharing	"I've detected a pattern: all Q3 milestones in Div Y slip by 2+ weeks"
alert	High-priority notification	"Budget utilization for Program X has exceeded 95% threshold"
handoff	Transfer task responsibility	"This compliance issue is outside my domain — routing to Governance"
request	Formal action request	"Please initiate intervention for Project Phoenix"

All A2A messages are persisted in a2a_conversations and a2a_messages tables, providing a complete audit trail of inter-agent collaboration.

3.6 Dual-Layer Cognitive Memory

The memory architecture is the key differentiator between Kyndryl Fabric [K360] agents and stateless AI assistants. Each agent maintains two layers of memory that give it continuity, pattern recognition, and institutional knowledge.

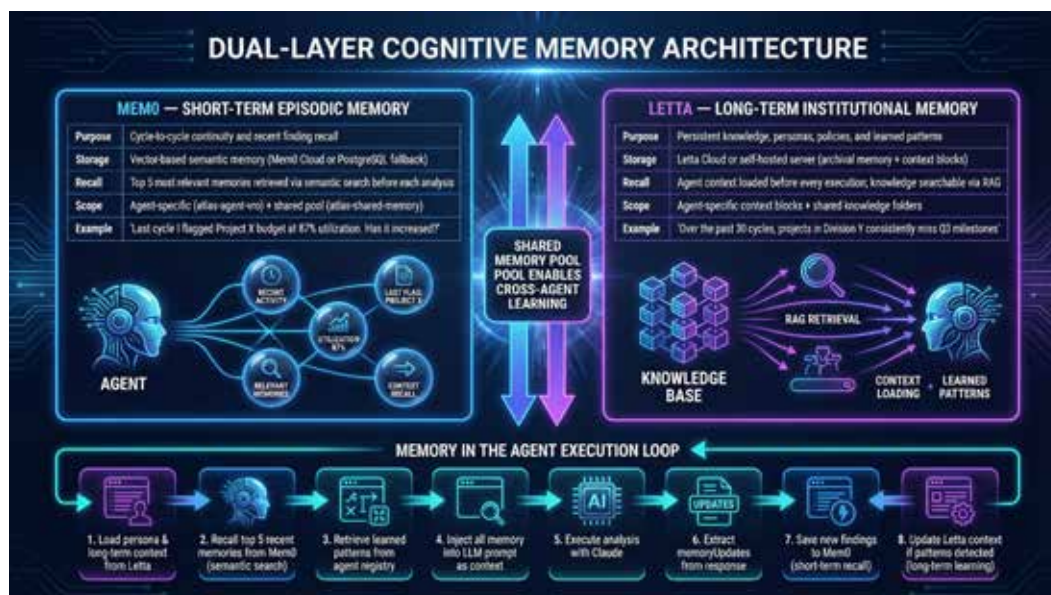


Figure 5: Dual-Layer Cognitive Memory Architecture (Mem0 + Letta)

Aspect	Mem0 — Short-Term Episodic	Letta — Long-Term Institutional
Purpose	Cycle-to-cycle continuity and recent finding recall	Persistent knowledge, personas, policies, and learned patterns
Storage	Vector-based semantic memory (Mem0 Cloud or PostgreSQL fallback)	Letta Cloud or self-hosted server (archival memory + context blocks)
Recall	Top 5 most relevant memories retrieved via semantic search before each analysis	Agent context loaded before every execution; knowledge searchable via RAG
Scope	Agent-specific + shared pool (atlas-shared-memory)	Agent-specific context blocks + shared knowledge folders
Example	"Last cycle I flagged Project X budget at 87% utilization. Has it increased?"	"Over the past 30 cycles, projects in Division Y consistently miss Q3 milestones"

4. Policy-as-Code — Governance as Executable Logic

4.1 The Problem with Traditional Policy Governance

In most enterprises, governance policies exist as PDF documents, SharePoint wikis, or slide decks. Compliance teams write them. Project teams ignore them. Auditors discover violations months later. The gap between "policy defined" and "policy enforced" is where governance failures occur.

Kyndryl Fabric [K360] eliminates this gap by treating policies as executable code that agents enforce autonomously at runtime. Every governance policy in the system exists in three forms simultaneously: human-readable natural language, machine-executable decision logic, and agent-enforceable behavioral constraints.

4.2 The Policy-as-Code Pipeline

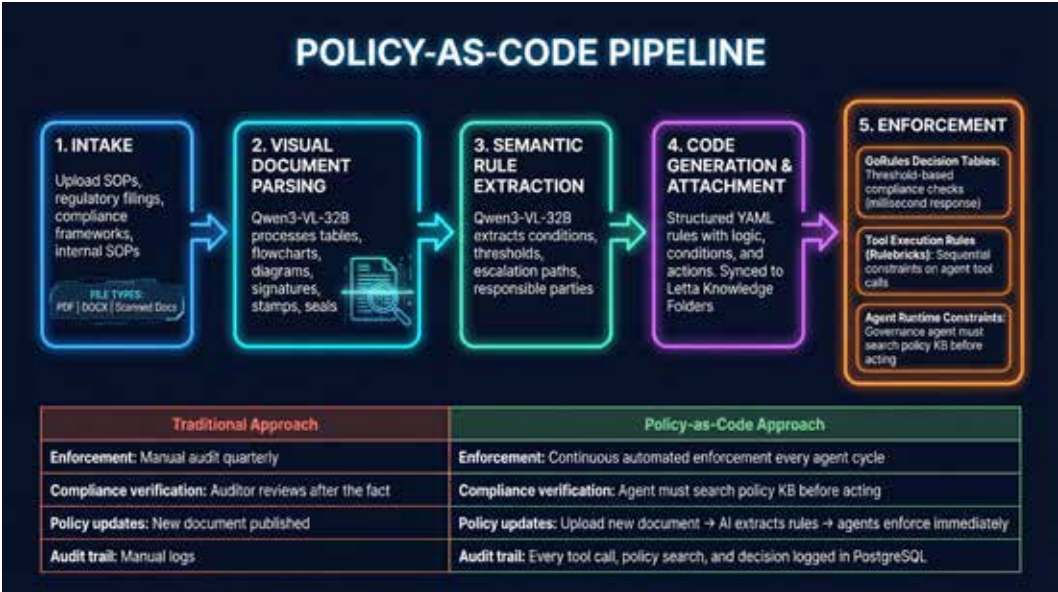


Figure 6: The Five-Stage Policy-as-Code Pipeline

4.3 Traditional vs. Policy-as-Code Comparison

Aspect	Traditional Approach	Policy-as-Code Approach
Enforcement	Manual audit (quarterly/annual)	Continuous automated enforcement (every agent cycle)
Compliance verification	Auditor reviews after the fact	Agent must search policy KB before acting (architecturally enforced)
Policy updates	New document published, teams notified	Upload new document → AI extracts rules → agents enforce immediately
Cross-domain application	Different teams interpret differently	Same policy applied consistently by all agents via shared knowledge
Audit trail	Manual logs	Every tool call, policy search, and decision logged in PostgreSQL

5. Temporal Knowledge Graph — Versioned Policy Intelligence

5.1 Why a Temporal Knowledge Graph

Traditional knowledge graphs represent the current state of relationships between entities. They answer "what is connected to what?" but not "how have those connections changed over time?" or "what was the policy context when a decision was made six months ago?"

Kyndryl Fabric [K360] employs a Temporal Knowledge Graph (TKG) — a time-versioned graph structure where every node, edge, and property carries a temporal dimension. This means the graph doesn't just store the current portfolio state — it stores every historical state, enabling agents to reason about change trajectories, policy evolution, and causal chains across time.

The Temporal Knowledge Graph serves three critical functions:

- 1. Policy versioning and lineage** — Every policy-as-code rule is a versioned node in the graph with full temporal lineage. Agents can query not just "what is the current budget threshold?" but "what was the threshold when Project Phoenix was approved, and how has it changed since?"
- 2. Causal reasoning across time** — By traversing temporal edges, agents can determine that a policy change in Q2 caused a cascade of compliance recalculations in Q3, which correlates with the delivery delays observed in Q4.
- 3. Memory integration** — The TKG is the bridge between short-term episodic memory (Mem0) and long-term institutional knowledge (Letta). Agent observations are anchored to temporal graph nodes, creating a structured knowledge fabric rather than a flat collection of memories.

5.2 Graph Structure and Temporal Dimensions

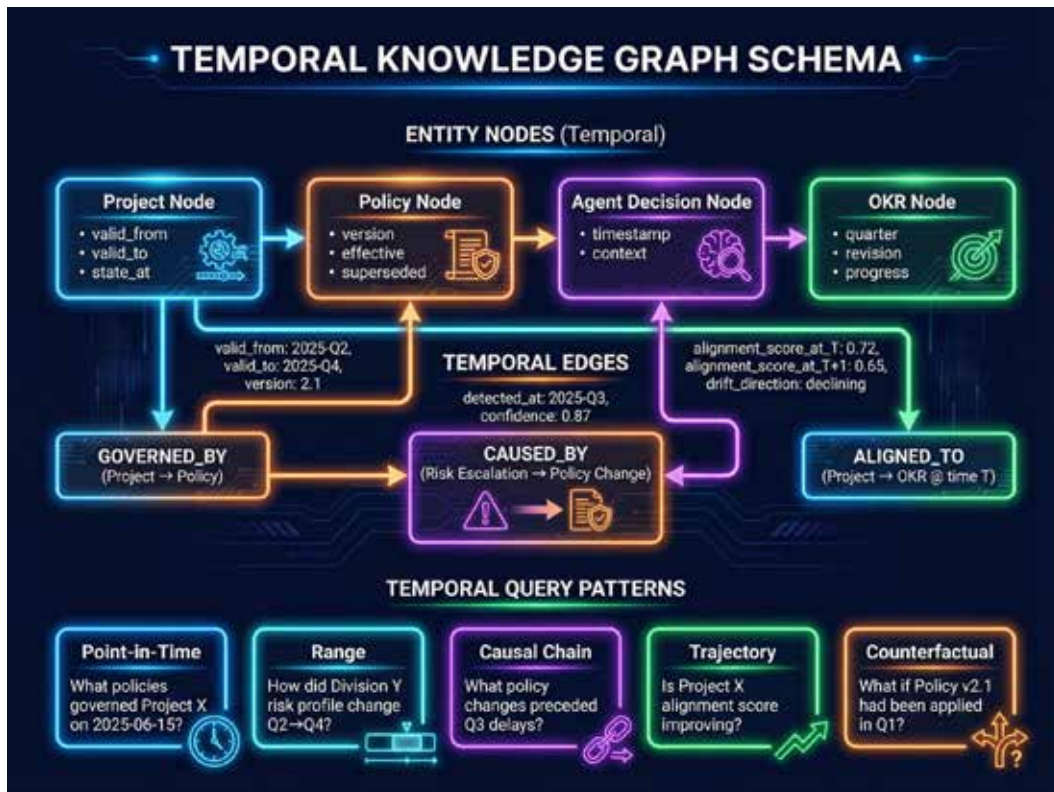


Figure 7: Temporal Knowledge Graph Schema — Nodes, Edges, and Query Patterns

5.3 Temporal Graph Query Patterns for Agent Reasoning

Agents use five classes of temporal queries that distinguish this system from flat memory architectures:

Query Type	Description	Agent Use
Point-in-Time	Snapshot of all entities and policies at a specific date	Retroactive compliance assessment — detect projects compliant when approved but non-compliant under current policy versions
Range	How metrics changed across a time period	Trend detection — acceleration patterns trigger proactive alerts before scores reach critical threshold
Causal Chain	What events preceded a specific outcome	Root cause identification — trace policy changes to downstream delivery delays
Counterfactual	What-if analysis on historical policy timing	Policy impact simulation — recommend optimal policy timing to avoid future overruns
Decay / Drift	Staleness detection for policies not re-evaluated	Governance hygiene — proactive identification of stale policies no longer reflecting current risk profiles

5.4 Temporal Graph and SAFe Ontology Integration

SAFe Entity	Temporal Dimension	Agent Use
-------------	--------------------	-----------

Portfolio	Investment allocation over time	Trend analysis: "Portfolio investment shifted 15% toward infrastructure over 3 quarters"
Value Stream	Flow metrics at each PI boundary	Velocity tracking: "Cloud Platform value stream throughput peaked in PI-4 and has declined since"
ART	Team composition and velocity per sprint	Capacity learning: "ART-7 consistently underperforms in sprints following resource rotations"
Epic	Status transitions with timestamps	Bottleneck detection: "Regulatory epics spend 3x longer in 'In Review' than infrastructure epics"
OKR	Progress snapshots per quarter	Alignment drift: "Key Result 3.2 was 72% aligned in Q1 but has drifted to 58% by Q3"
Policy	Version lineage with supersession edges	Policy effectiveness: "Threshold tightening in v2.0 reduced false escalations by 40% but increased approval latency by 3 days"
Risk	Severity trajectory over time	Acceleration detection: "Risk R-14 severity is now accelerating — exponential projection crosses critical in 6 weeks"
Intervention	Outcome tracking (approved/rejected/modified)	Learning signal: "Interventions of type 'budget pause' are rejected 60% of the time for committed-date projects"

6. Autonomous Learning Architecture — How Agents Develop Expertise

6.1 Beyond Stateless AI: Why This Is a Deep Learning Architecture

Most enterprise AI systems are stateless. They receive a query, generate a response, and forget everything. Each interaction starts from zero. This is fundamentally different from how human expertise develops — through accumulated experience, pattern recognition, and learning from outcomes.

Kyndryl Fabric [K360] implements the core principles of deep reinforcement learning — continuous observation, action, reward, and adaptation — through a memory-augmented LLM architecture rather than traditional neural network training. The agents don't train gradient descent on GPUs. Instead, they: Observe fresh portfolio data every cycle, Act by generating insights and recommendations, Remember findings in dual-layer memory, Learn from human feedback on interventions, and Adapt so that future analysis cycles incorporate all accumulated memory.

6.2 Deep Learning Principles Comparison

Deep Learning Principle	Traditional Implementation	K360 Implementation
Memory	LSTM/Transformer attention over sequences	Dual-layer memory (episodic Mem0 + archival Letta) over analysis cycles
Reward Signal	Loss function / reward function	Human intervention feedback (approve/reject)
Adaptation	Gradient descent updates weights	Memory-informed prompting adjusts agent behavior
Multi-Task Learning	Shared representations across tasks	Shared memory pool enables cross-agent knowledge transfer
Transfer Learning	Pre-trained model fine-tuned for domain	Claude's general knowledge fine-tuned via Letta persona + domain-specific memory
Continuous Learning	Online learning / continual learning	Every analysis cycle accumulates new knowledge without catastrophic forgetting

The key insight: Kyndryl Fabric [K360] achieves continuous, autonomous, self-improving analysis without the infrastructure cost of GPU clusters, model training pipelines, or ML ops. The "learning" happens through memory accumulation and retrieval, not weight updates — but the behavioral outcome is the same: agents that get smarter over time.

7. The GoRules Decision Engine — Deterministic Governance

7.1 Architecture Rationale

AI agents provide qualitative reasoning, but they have limitations: non-deterministic outputs, API latency, and per-call costs. For quantitative threshold monitoring — "is this metric above or below a boundary?" — a deterministic rules engine is faster, cheaper, and more reliable.

The GoRules ZEN Engine provides this deterministic layer. It evaluates threshold rules in milliseconds with zero API cost, ensuring governance monitoring never depends on external service availability.

7.2 Rule Categories

Category	Count	Examples
Threshold Rules	42	Budget variance escalation, velocity drop detection, risk score boundaries
Agent Behavior Rules	Variable	Event routing (budget_breach → FinOps, compliance_alert → Governance)
Workflow Rules	Variable	Multi-step escalation sequences (Red status → PMO intervention → TMO root cause → VRO value impact)
GDPR Compliance	Variable	Data privacy rules, consent tracking, sovereignty enforcement

7.3 Action Execution

When a rule fires, the Action Executor maps the output to concrete actions:

Action	Effect
send_a2a_task	Dispatches a task to another agent's queue
broadcast_notification	Pushes real-time alert to all connected dashboards
store_memory	Persists the finding in an agent's Mem0 memory
chain_rule	Triggers evaluation of another rule (cascading logic)
create_intervention	Creates a pending human decision in the intervention queue

This means the rules engine and AI agents share the same action vocabulary. A threshold breach can trigger the same A2A task that an agent's LLM reasoning would — blurring the line between deterministic and intelligent governance.

8. The Integration Fabric — MCP Server & Write-Back

8.1 Design Philosophy

The platform operates as a governance overlay, not a replacement for existing tools. Teams continue using their preferred systems (Jira for sprints, OpenProject for waterfall, Confluence for documentation, ServiceNow for ITSM). Kyndryl Fabric [K360] provides the unified portfolio view that no single tool can deliver on its own.

The integration philosophy is "sync, don't migrate." Data flows bidirectionally: Inbound sync engines and webhook handlers pull data from external systems, map it to the SAFe 6.0 ontology, and store it in PostgreSQL. Outbound, the Write-Back API pushes governed changes, AI-generated insights, and intervention outcomes back to source systems.

8.2 Supported External Systems (12)

System	Client Status	Sync Capabilities
OpenProject	Production-ready	Full bidirectional sync, cost tracking, category mapping
Jira / Atlassian	Production-ready	Projects, epics, stories, sprints, story points
Confluence	Production-ready	Spaces, pages, search, insight report publishing
ServiceNow	Production-ready	Change requests, incidents, CMDB integration
Azure DevOps	Production-ready	Work items, boards, repos, pipelines
Plane.so	Production-ready	Issues, cycles, modules with prefix-based entity mapping
Planview	API client ready	Portfolios, programs, resources
Monday.com	API client ready	Boards, items, updates
Rally / Broadcom	API client ready	User stories, iterations, releases
Asana	API client ready	Projects, tasks, sections
Smartsheet	API client ready	Sheets, rows, columns
MS Project Online	API client ready	Projects, tasks, resources

8.3 Write-Back System

The Write-Back API (POST /api/writeback) enables governed change propagation. AI agents detect an issue → generate a recommendation → leadership approves via the dashboard → the approved change is pushed back to the source system automatically. This closes the loop between AI analysis and operational action.

Entity Type	Target Systems	Actions
Project	Jira, OpenProject, Plane, Confluence	Create, update, comment
Epic	Jira, Plane	Create, update
Insight	Confluence	Create (AI insight reports as wiki pages)
Risk	Plane	Create, update
Task / Story	Plane	Create, update

9. Executive Intelligence Layer

9.1 Executive Insights Engine

The Executive Insights Engine generates structured leadership briefings by aggregating data across eight dimensions: Strategic Themes and Value Streams, Epic and ART velocity, Division financial performance, Project portfolio health (RAG distribution), VRO benefit realization metrics, PMO delivery execution metrics, Enterprise risk register, and Compliance checkpoint status.

The engine constructs a comprehensive context document from PostgreSQL data, then prompts Claude to generate a structured executive briefing with quantified impacts (in EUR/USD), evidence linked to specific project IDs, and prioritized recommendations. An MD5 hash of the data context is computed before each generation — if the hash matches the previous run, the cached briefing is returned immediately (zero API cost, zero latency).

9.2 Insight Synthesizer — The Meta-Agent

The Insight Synthesizer operates above the individual agents as a cross-portfolio pattern detection system. It correlates findings across all nine agents to surface systemic patterns invisible to any single domain specialist.

Capability	Description
Pattern Detection	Evaluates configurable risk patterns (e.g., "Budget Burn vs. Milestone Delay") against real-time metrics
Root Cause Analysis	Uses graph traversals to identify dependency cascades, circular dependencies, and correlated risks
Autonomy Levels	Three-tier action system: advisory (recommendations only), supervised (human-in-the-loop for critical decisions), full (agent executes autonomously via A2A)
Synthesis	Transforms technical pattern matches into natural language executive summaries

10. Data Architecture

10.1 Database Design

PostgreSQL with 105 tables managed by Drizzle ORM. The schema follows the SAFe 6.0 ontology with extensions for AI agent operations:

Domain	Tables	Key Entities
SAFe Hierarchy	15	Portfolios, Value Streams, ARTs, Teams, Program Increments, Sprints
Work Items	8	Strategic Themes, Epics, Features, Stories, Tasks, Milestones, Dependencies
Financial	4	Project Financials, Cost Tracking, Benchmarks, Spend Records
OKR & Strategy	5	OKRs, Key Results, Division KPIs, Division OKRs, Division Risks
Risk & Compliance	6	Risks, Risk Patterns, Compliance Checkpoints, GDPR Rules, Threshold Rules, Audit Logs
Agent Operations	10	Agent State, Agent Attributes, Activity Log, A2A Conversations, A2A Messages, Interventions
Integration	8	Sync Jobs, Sync Runs, Webhook Endpoints, Webhook Events, Ingestion Sessions, QA Reviews
Resources	4	Resources, Stakeholder Readiness, Team Metrics, Resource Allocations
Other	45	Users, Sessions, Dashboard Widgets, Templates, VRO Metrics, Token Usage, and more

10.2 Schema as Single Source of Truth

All schema definitions live in `shared/schema.ts` and generate three artifacts from each table definition: (1) Drizzle table schema for type-safe database queries, (2) Zod insert schema for API request validation, and (3) TypeScript types for compile-time safety across frontend and backend. A schema change in one place automatically propagates to validation rules, API types, and database queries — eliminating the drift between data model, API contract, and UI types.

11. Frontend Architecture

11.1 Technology Stack

Technology	Purpose	Rationale
React 18	UI framework	Component-driven architecture, mature ecosystem, TypeScript support
TypeScript	Type safety	Compile-time error detection across 21 pages and 470+ API integrations
Wouter	Routing	1.5KB alternative to React Router; hook-based routing without data loader overhead
TanStack React Query	Server state	Automatic caching, background refetch, stale-while-revalidate, request deduplication
Tailwind CSS v4	Styling	Utility-first CSS with design token system for consistent corporate identity
shadcn/ui + Radix UI	Components	Accessible, keyboard-navigable UI primitives (WCAG 2.1 compliant)
Recharts	Visualization	Portfolio health gauges, risk heat maps, flow metric trends, financial burn-down charts
Framer Motion	Animation	Page transitions and micro-interactions for responsive feel
Monaco Editor	Code editing	Same engine as VS Code; used for rules editor and policy generator

11.2 Dashboard Portfolio (21 Pages)

Page	Audience	Description
Executive Dashboard	C-Suite	AI-generated narrative insights, cross-portfolio risk summary, strategic alignment scores
PMO Dashboard	Program Managers	Project health indicators, milestone tracking, resource utilization, dependency graphs
TMO Dashboard	Transformation Office	Change readiness scores, adoption curves, benefits realization timelines
FinOps Dashboard	Finance	Budget vs. actual, cost burn-down, forecast accuracy, optimization recommendations
OKR Dashboard	Strategy	OKR cascade visualization, alignment scoring, orphaned project detection
Governance Dashboard	Compliance	Compliance checkpoints, policy adherence, audit trail, regulatory status
Planning Dashboard	Planning Office	Strategic roadmaps, capacity planning, resource forecasting, PI planning
OCM Dashboard	Change Management	Stakeholder readiness, training completion, adoption barriers, change impact
Agent Command Center	IT / Operations	Live monitoring of all 9 agents — status, findings, memory, A2A messages
Agent Config Hub	Administrators	Agent behavior configuration — frequency, sensitivity, personas, data domains
Simulation Sandbox	Leadership	What-if scenario modeling with agent re-run and write-back to external systems
Rules Editor	Governance	Visual rule builder with Monaco editor for threshold rule definitions
MCP Config	Integration	Connection management, field mapping, sync scheduling, error logs for all external systems

Risk Center	Risk Management	Enterprise risk heat maps, proximity analysis, pattern detection, impact mapping
Flow Metrics	Agile Coaches	SAFe flow metrics (Distribution, Velocity, Time, Load, Efficiency) with anomaly detection
Project Ingestion	Data Stewards	AI-powered 5-step import wizard with Claude-assisted field mapping and QA gate
Insight Synthesizer	Leadership	Cross-agent pattern correlation and root cause analysis interface
Policy Generator	Legal / Compliance	AI-assisted governance policy document creation and analysis
VRO Framework	Value Office	Value stream visualization, benefit maps, investment portfolios, ROI tracking
Sustainability	ESG Officers	Carbon footprint per project, climate risk exposure, sustainability alignment scoring
A2A Communication	IT / Operations	Timeline view of inter-agent messages, skill invocations, collaboration sessions

12. Security & Access Control

12.1 Authentication

Dual-layer authentication: (1) OAuth — Standard user login for dashboard access with PostgreSQL-backed sessions (express-session + connect-pg-simple). (2) Password-based admin — Separate admin credentials for agent configuration, rule editing, and system administration.

12.2 Authorization (RBAC)

Role	Access Level
Admin	Full access to all features including agent configuration and rule editing
PM	Project management features, simulation, and write-back
Viewer	Read-only dashboard access

12.3 Data Privacy

GDPR compliance engine enforces data privacy rules across the platform, managing consent tracking and ensuring personal information in stakeholder and resource records is handled according to regulatory requirements.

13. Cost Management & Operational Efficiency

13.1 AI API Cost Controls

Mechanism	Impact
Inactivity pause	Agents stop when no users connected (10-min threshold). Eliminates off-hours API spend.
Data-change gating	Skip AI analysis when data unchanged. Reduces calls by 40–60% during stable periods.
Round-robin execution	Only 3 agents per cycle (not all 9). Predictable per-cycle cost.
Claude Haiku 4.5	Lowest-cost model with sufficient reasoning capability. \$0.25/MTok input, \$1.25/MTok output.
Structured output	JSON-only responses eliminate conversational overhead. Reduces token usage per call.
Executive insight caching	MD5 hash comparison skips regeneration when data unchanged.

14. Technology Stack Summary

14.1 Core Platform

Layer	Technology	Version
Frontend Framework	React	18
Language	TypeScript	5.x
Backend Runtime	Node.js	20
Backend Framework	Express.js	4.x
Database	PostgreSQL (Neon)	16
ORM	Drizzle	Latest
Schema Validation	Zod	3.x

14.2 AI & Agent Stack

Component	Technology	Purpose
Agent Framework	Mastra 1.5	Agent definitions, tool registration, execution orchestration
Agent Orchestration	LangGraph	Stateful multi-agent workflow graphs, conditional branching, human-in-the-loop
Chain Composition	LangChain	LLM chain construction, prompt templating, RAG pipelines
Observability	LangSmith	Agent trace logging, prompt evaluation, latency profiling
LLM Provider (Agents)	Anthropic Claude Haiku 4.5	Agent reasoning, analysis, structured output generation
Vision-Language Model	Qwen/Qwen3-VL-32B	Policy document ingestion — SOP parsing, regulatory filing extraction
Knowledge Graph	Neo4j	Temporal Knowledge Graph — entity relationships, causal chains, temporal traversals
Short-Term Memory	Mem0	Episodic memory, semantic recall, cross-agent shared memory
Long-Term Memory	Letta	Persistent knowledge, policy storage, RAG, tool rules
Agent Communication	A2A Protocol (custom)	Peer-to-peer task delegation, messaging, handoffs
Decision Engine	GoRules ZEN Engine	Deterministic threshold monitoring, JDM decision tables

15. What Makes This Architecture Unique

15.1 Compared to Traditional Enterprise Dashboards

Capability	Traditional Dashboard	Kyndryl Fabric [K360]
Data Freshness	Batch refresh (hourly/daily)	Continuous agent analysis + real-time sync
Anomaly Detection	Manual analyst review	Autonomous AI agents detect and alert proactively
Cross-Domain Analysis	Separate tools, separate teams	A2A protocol enables automatic cross-agent collaboration
Institutional Memory	Analyst turnover = knowledge loss	Dual-layer memory (Mem0 + Letta) preserves institutional knowledge
Policy Enforcement	PDF policies, periodic manual audits	Policy-as-Code: executable rules enforced at agent runtime
Learning	Analysts build expertise over years	Agents accumulate institutional knowledge over cycles, learn from human feedback
Multi-Tool Portfolio View	Manual consolidation from Excel	Automated sync with 12 PPM tools via MCP
What-If Analysis	Spreadsheet modeling	Simulation sandbox with AI agent re-run
Action Execution	Manual updates in each tool	Write-back API propagates decisions to source systems

15.2 Key Architectural Differentiators

- 1. Agents that learn and remember** — The dual-layer memory system (Mem0 + Letta) creates agents with genuine continuity. An agent running for a month has accumulated expertise that a fresh instance would lack.
- 2. Policy-as-Code as runtime constraint** — Governance policies are not documentation — they are executable code. The Governance agent architecturally cannot act without first consulting its policy knowledge base.
- 3. Ontology-integrated learning** — Agent memories are anchored to the SAFe 6.0 ontology. When an agent learns a pattern, it learns it in context: which Value Stream, which ART, which Division, which Strategic Theme.
- 4. Agents that collaborate emergently** — The A2A protocol enables multi-agent workflows that are not pre-programmed. When one agent's analysis reveals cross-domain implications, it autonomously initiates collaboration with peer agents.
- 5. Deterministic + probabilistic governance** — The combination of GoRules (fast, cheap, reliable, auditable) and Claude (nuanced, contextual, reasoning) provides governance that is both trustworthy and intelligent.
- 6. Closed-loop from insight to action** — The pipeline from detection → analysis → recommendation → approval → write-back → source system update eliminates the "last mile" gap where insights go unactioned.
- 7. Temporal Knowledge Graph for causal reasoning** — The TKG adds a time dimension to every entity, relationship, and policy. Agents can traverse temporal edges to determine why

things changed, trace causal chains across quarters, and detect policy drift before it causes downstream failures.

8. Governance overlay, not replacement — The MCP integration approach means organizations adopt K360 without disrupting existing tool chains. Teams keep using Jira, OpenProject, and Confluence while gaining portfolio-level AI governance across all of them simultaneously.

16. Agentic Backbone — Enterprise Integration via ACP and A2A

16.1 The Agentic Backbone Concept

Kyndryl Fabric [K360] is not a closed system. It is designed to operate as an Agentic Backbone — a central nervous system for enterprise AI that other platforms, departments, and organizations can connect their own agents to. This is the critical architectural distinction between "an application with AI features" and "an AI-native platform that participates in an enterprise agent mesh."

The Agentic Backbone is built on two open protocols:

A2A (Agent-to-Agent Protocol) — Google's open standard for inter-agent communication, enabling agents built on different frameworks, by different teams, running in different environments, to discover each other's capabilities, delegate tasks, exchange structured messages, and maintain multi-turn conversations.

ACP (Agent Communication Protocol) — IBM/BeeAI's protocol for standardized agent interoperability, providing a uniform interface for agent registration, capability advertisement, and cross-platform task orchestration.

16.2 Architecture — Kyndryl Fabric [K360] as Agentic Backbone

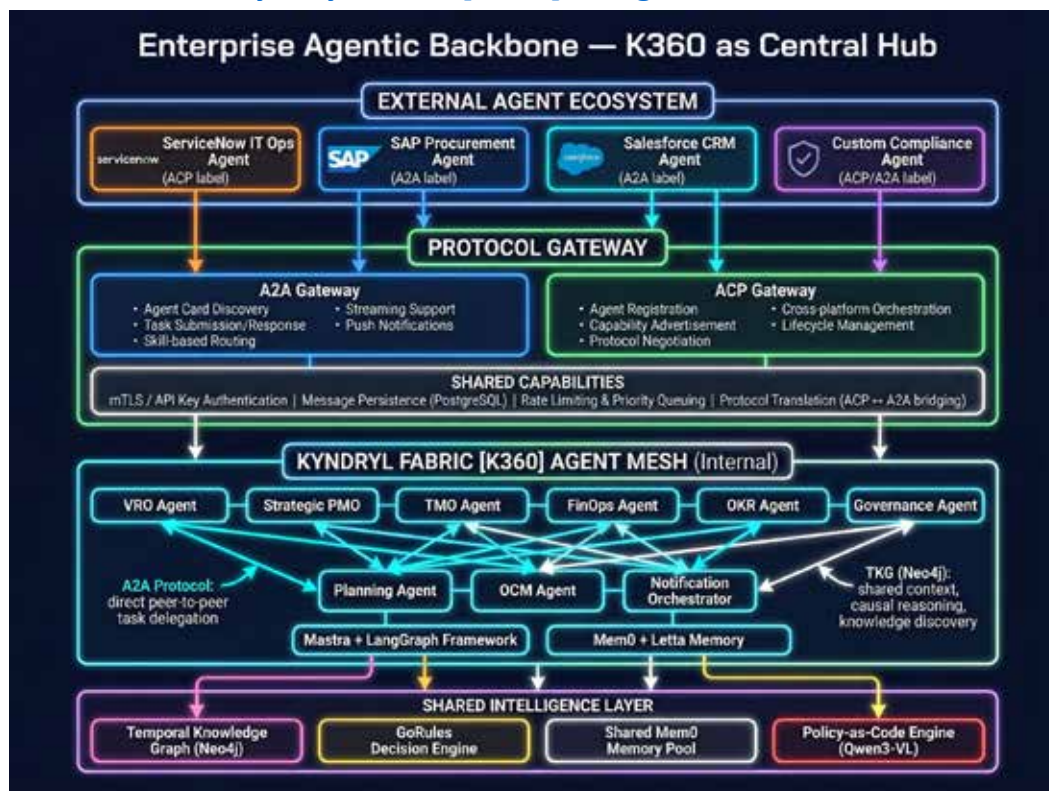


Figure 8: Enterprise Agentic Backbone — K360 as Central Intelligence Hub

16.3 A2A Protocol — Agent Discovery and Task Delegation

Kyndryl Fabric [K360] implements the A2A protocol through its Agent Card system, enabling any external agent to discover, communicate with, and delegate tasks to K360 agents. Every K360 agent publishes an Agent Card — a machine-readable manifest that advertises its identity, capabilities, and communication endpoints.

External agents discover K360 agents through a standard discovery endpoint (GET /api/a2a/agents), filtering by skill or domain. This means a ServiceNow IT Operations agent can query K360's discovery endpoint, find the Governance Guardian agent, and request a compliance assessment — without any custom integration code.

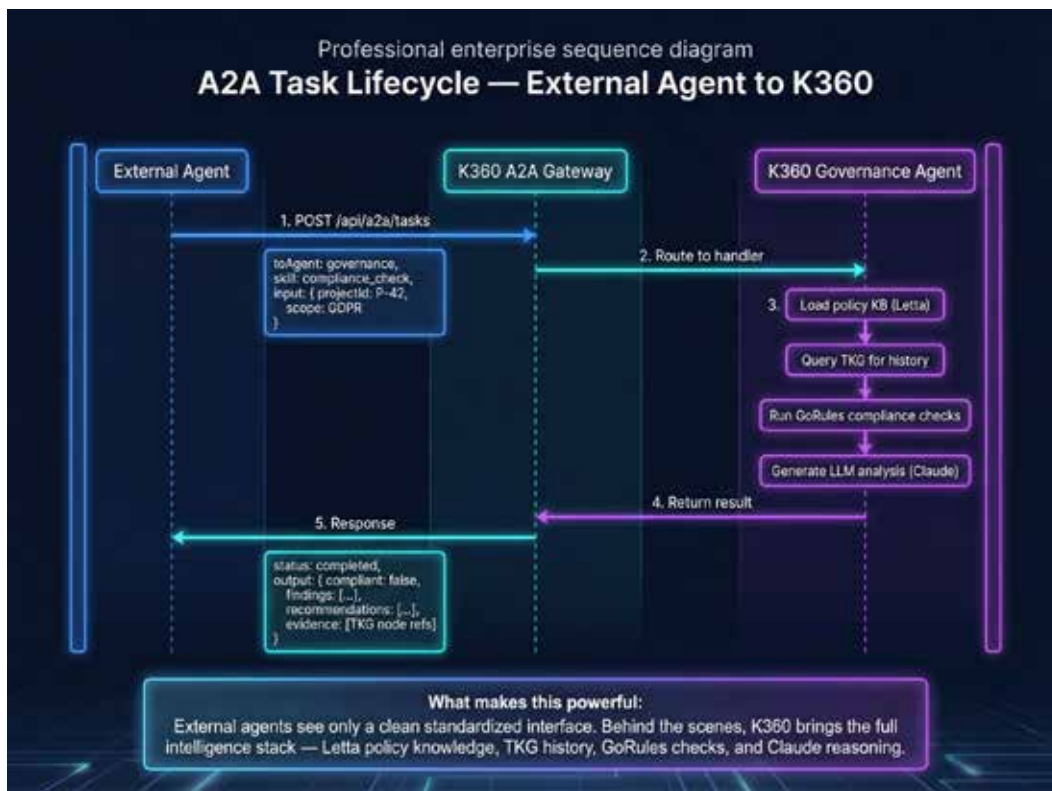


Figure 9: A2A Task Lifecycle — External Agent to K360 Governance Agent

16.4 ACP Protocol — Cross-Platform Agent Orchestration

While A2A handles point-to-point agent communication, ACP provides the higher-level orchestration layer for multi-platform agent workflows. External agents register with the K360 backbone through a standardized registration flow, advertising their capabilities and authentication credentials. ACP then manages the full lifecycle of external agent participation.

ACP Lifecycle Stage	Function	Description
Registration	register	External agent joins the backbone with capabilities and authentication
Discovery	discover	Agents find each other by capability, domain, or platform
Health Check	heartbeat	Periodic liveness checks ensure registered agents are still available

Capability Update	update	Agents can advertise new capabilities or deprecate old ones
Deregistration	deregister	Clean removal from the backbone with graceful task handoff
Protocol Bridge	negotiate	Determines the best communication protocol between two agents

16.5 What the Agentic Backbone Enables

Capability	Without K360 Backbone	With K360 Backbone
Cross-platform agent communication	Manual API integration per system pair	Standardized A2A/ACP protocols — connect once, communicate with all
Agent discovery	Hardcoded endpoints and capabilities	Dynamic discovery via Agent Cards and ACP registry
Multi-platform workflows	Custom orchestration code for each workflow	Agents discover capabilities and compose workflows dynamically
Governance across platforms	Each platform enforces its own policies	K360 Governance agent applies unified policy-as-code across all participating agents
Institutional memory	Siloed per platform	Shared memory pool (Mem0) + Temporal Knowledge Graph accessible to all backbone participants
Audit trail	Fragmented across systems	Unified audit trail in K360 — every cross-platform interaction logged with TKG temporal edges

16.6 Security Model for the Agentic Backbone

Security Layer	Implementation
Transport	Mutual TLS (mTLS) for all backbone communication — both sides authenticate
Authentication	API key or certificate-based authentication at registration; rotated on configurable schedule
Authorization	Capability-based access control — external agents can only invoke skills they are authorized for
Data Classification	Message payloads tagged with classification levels; the backbone enforces that classified data only flows to agents with matching clearance
Rate Limiting	Per-agent rate limits prevent any single external agent from overwhelming the backbone
Audit	Every cross-backbone message persisted in PostgreSQL with full metadata (sender, receiver, timestamp, payload hash, outcome)
Policy Enforcement	External agent requests pass through the same GoRules + Letta policy enforcement as internal agent actions

16.7 Why ACP + A2A Together

Concern	A2A Alone	ACP Alone	A2A + ACP Together
Agent Discovery	Agent Cards with skill lists	Capability registry with versioning	Agent Cards for skill-level discovery + ACP registry for platform-level management
Task Execution	Direct task submission and response	Orchestrated multi-step workflows	A2A handles the task exchange; ACP handles the orchestration and routing

Protocol Flexibility	Single protocol only	Single protocol only	ACP negotiates the best protocol for each agent pair — may use A2A, REST, gRPC, or messaging
Lifecycle Management	No lifecycle management	Registration, health checks, deregistration	ACP manages the backbone membership; A2A handles the conversations
Cross-Platform	Works if both sides implement A2A	Works if both sides implement ACP	ACP bridges between agents that use different protocols; A2A provides the common exchange format

The combination means Kyndryl Fabric [K360] can participate in any enterprise agent ecosystem — regardless of whether the other agents use Google's A2A, IBM's ACP, Microsoft's AutoGen, LangChain agents, CrewAI, or custom implementations. The protocol gateway translates between them, and the K360 agents bring their full intelligence (memory, policies, temporal graph, deterministic rules) to every interaction.

Document Notes

This document describes the architecture as designed and deployed. The A2A protocol is implemented and operational in the current production environment (Agent Cards, task protocol, discovery). ACP integration is architecturally designed and ready for implementation — the protocol gateway pattern is proven through the existing A2A infrastructure.

Version: 3.0 | Date: March 2026 | Classification: Confidential — Technical Reference